Analysis of different MOR methods for solving nonlinear Elliptic PDEs

1st Maddalena Ghiotti s332834 | 2nd Valerio Taralli s332689 | 3rd Gaia Saroglia s332687

Abstract

PUNTI DA SCRIVERE (cenni):

- ◇ **cosa facciamo e di cosa si tratta**
- ◇ **descrizione problema (su cosa facciamo esperimenti numerici): problema ellittico non lineare parametrizzato, condizioni al contorno, configurazioni in analisi**
- ◇ **obiettivo: valutare e confrontare POD e PINNs+POD-NN rispetto a full order**
- ◇ **risultati: cosa abbiamo ottenuto dalle simulazioni**

I. Introduction

The numerical approximation of nonlinear parametrized partial differential equations (PDEs) using high fidelity techniques, which means solving the full discretized problem, called full-order model (FOM), typically results in very large systems that become computationally expensive and often unfeasible. For this reason, model order reduction (MOR) techniques have gained considerable attention: they are exploited to decrease computational costs while maintaining a good solution approximation.

In the present paper, we study a nonlinear elliptic parametrized equation with homogeneous Dirichlet boundary conditions, on a two-dimensional square domain: both cases with source term dependent on or independent of the parameter are considered. In these two settings, three approaches are implemented[1]:

1. the first is the POD-Galerkin method over a finite element FOM;
2. then we focus on parametric PINN, using the same network structure with either parameters independent and dependent source term;
3. in the end, we use, as another reduce method, the hybrid approach of the POD-NN.

Afterwards, the results in terms of computational costs and accuracy are compared using as a benchmark the FOM results.

The outline of the project is as follows: we first introduce the model definition and the methods used; the numerical studies, results and comparisons are next tackled for the given problems; lastly, the conclusions are discussed.

[1]For the acronyms meaning see later sactions.

II. Model definition and methods

The nonlinear elliptic PDE studied consist of finding

$$u(\mathbf{x};\boldsymbol{\mu}):\Omega\times\mathcal{P}\to\mathbb{R}$$

such that it is the solution of the following parametrized problem with homogeneous Dirichlet boundary conditions:

$$\begin{cases} -\Delta u + h = g & \text{in } \Omega\times\mathcal{P}, \\ u = 0 & \text{on } \partial\Omega\times\mathcal{P}, \end{cases} \tag{1}$$

where

$$h(u;\boldsymbol{\mu}) \equiv \frac{\mu_0}{\mu_1}(e^{\mu_1 u(\mathbf{x};\boldsymbol{\mu})}-1):\Omega\times\mathcal{P}\to\mathbb{R}$$

is the nonlinear term,

$$g(\mathbf{x};\boldsymbol{\mu}):\Omega\times\mathcal{P}\to\mathbb{R}$$

is the source term to be defined,

$$\mathbf{x}\equiv(x_0,x_1)\in\Omega=(0,1)^2$$

is the spatial square domain and

$$\boldsymbol{\mu}\equiv(\mu_0,\mu_1)\in\mathcal{P}=[0.1,1]^2\subset\mathbb{R}^2$$

is the parametric square domain. For the source term we consider two different cases:

C1. independent of $\boldsymbol{\mu}$ with $g(\mathbf{x};\boldsymbol{\mu})\equiv g_1(\mathbf{x})$ where:

$$g_1(\mathbf{x}) \equiv 100\sin(2\pi x_0)\cos(2\pi x_1) \tag{2}$$

C2. dependent on $\boldsymbol{\mu}$ with $g(\mathbf{x};\boldsymbol{\mu}) \equiv g_2(\mathbf{x};\mu_0)$ where:

$$g_2(\mathbf{x};\mu_0) \equiv 100\sin(2\pi\mu_0 x_0)\cos(2\mu_0\pi x_1) \quad (3)$$

- POSSIBLE FIGURE ABOUT THE DOMAIN -

In the following, we briefly describe the three approaches studied to solve problem (1), which will be compared in terms of computational costs and accuracy with respect to the full order model in Section III.

A. Proper Orthogonal Decomposition (POD)

POD is known to lead to fast and reliable evaluations of the solution for new parameter values by building the reduced space; in the affine parameter case POD allow us to decouple the computation in offline and online phases too, reducing even greatly the computational costs.

Considering a parametrized partial differential equation whose solution lies in the functional space $\mathbb{V}^{\boldsymbol{\mu}}$, POD is an algorithm for the generation of the reduced basis based on two different stages:

1. we explore the information related to the solution that varies with respect to $\boldsymbol{\mu}$ in a finite dimensional set $\mathcal{P}_M \subset \mathcal{P}$ of dimension M.
2. we compress the information and we build a linear subspace $\mathbb{V}_N \subset \mathbb{V}_\delta$ of dimension $N \ll N_\delta$.

Building the space and storing the $\boldsymbol{\mu}$-independent quantities is the offline phase, which depends on the dimension of the original discrete model. Once the space is built, a fast online phase occurs, where we can compute many solutions in real time for new instances of the parameter input.

In case of a linear PDE with an affine dependence on the parameters, one can easily decouple the online step from the full-order numerical scheme, so that the cost of any online query is uniquely related to the dimension of the reduced space by means of the affine decomposition. However, for differential problems with a non affine parametric dependence one needs some further actions to guarantee a significant computational saving with respect to the underlying full order model.

In the POD method, to find the basis for our reduced space, we need to define the correlation matrix $\mathbf{C} \in \mathbb{R}^{M\times M}$. First, we select M solutions $\omega_m = u_\delta(\boldsymbol{\mu}_m) \in \mathbb{V}_\delta$ from the high fidelity problem. Then we solve the eigenvalue problem:

$$\mathbf{C}\alpha_m = \lambda_m \alpha_m \quad \text{for } m \in \{1,\cdots,M\}. \quad (4)$$

The correlation matrix has a good property: it is symmetric positive semi-definite. So, we can order the all-positive eigenvalues as $\lambda_1 \geq \cdots \geq \lambda_M \geq 0$ and retain the first N eigen-pairs (λ_n,α_n) for $1 \leq n \leq N$. In this way, we obtain the linear independent basis function that we use to build the reduced space. Indeed, having all the ingredients defined above, we can compute:

$$\chi_n = \sum_{m=1}^{M} (\alpha_n)_m \omega_m, \qquad 1 \leq n \leq N, \quad (5)$$

and normalize them, obtaining $\psi_n = \dfrac{\chi_n}{||\chi_n||_{\mathbb{V}}}$.

B. Physics-Informed Neural Networks (PINN)

In nonlinear frameworks, to overcome some of the limitations of the POD method and to exploit the advantages of the division in online and offline phases, the recent advances in machine learning have opened new opportunities for solving PDEs. In particular, PINN is a flexible approach that directly uses the equations into the training process of a neural network, it incorporates physical laws given by the differential equations into the loss functions to take advantages in the learning process. This allows PINNs to deal with complex and parametric problems in a data efficient and mesh free way.

Having a general parametric nonlinear partial differential equation, PINNs are neural networks that are trained to solve supervised learning tasks while respecting the laws of physics described by the given equations. The main idea is to incorporate into the loss function both the information obtained from the data and the physical model. The physics-informed term is like a prior knowledge that provides more information to the PINN without the need for more measurements, but only by evaluating the residual of the PDE at additional points in the domain. In our case, we want to approximate the solution of the PDE by considering two contributions to the loss function: one related to boundary information and the other to residual information.

The general structure of the parametric PINN method can be summarized as follows:

$$\begin{aligned} \mathcal{F}_{NN}: \quad & \Omega \times \mathcal{P} \to \mathbb{V}^{\boldsymbol{\mu}}, \\ & (x,\boldsymbol{\mu}) \mapsto \hat{u}(x,\boldsymbol{\mu}) \approx u(x,\boldsymbol{\mu}). \end{aligned} \quad (6)$$

where $u(x,\boldsymbol{\mu})$ is the exact solution of the PDE and $\mathbb{V}^{\boldsymbol{\mu}}$ is the functional space where the solution lies. PINNs use optimization algorithms to update the network parameters until the value of the loss function is decreased to a prespecified level.

Given the neural network $v(x,\boldsymbol{\mu}) \in \mathcal{F}_{NN}$, the loss function is defined as:

$$\text{MSE} = \text{MSE}_b^{\boldsymbol{\mu}} + \lambda \text{MSE}_p^{\boldsymbol{\mu}} \tag{7}$$

where $\text{MSE}_b^{\boldsymbol{\mu}}$ is the boundary loss, $\text{MSE}_p^{\boldsymbol{\mu}}$ is the physical loss and λ is the hyperparameter that balances the tradeoff between the two contributions, to be set before the network training.

In particular, if the physical system we are studying is described by a PDE of the form $\mathcal{R}(u(x,\boldsymbol{\mu})) = 0$, the errors are given by:

$$\text{MSE}_b^{\boldsymbol{\mu}} = \frac{1}{N_b} \sum_{k=1}^{N_b} |v(x_k^b, \boldsymbol{\mu}_k^b) - u(x_k^b, \boldsymbol{\mu}_k^b)|^2,$$

$$\text{MSE}_p^{\boldsymbol{\mu}} = \frac{1}{N_p} \sum_{k=1}^{N_p} |\mathcal{R}(v(x_k^p, \boldsymbol{\mu}_k^p))|^2$$

for $(x_k^b, \boldsymbol{\mu}_k^b) \in (\partial\Omega, \mathcal{P})$ and $(x_k^p, \boldsymbol{\mu}_k^p) \in (\Omega, \mathcal{P})$.

Finally, the solution approximation is given by

$$\hat{u}(x,\boldsymbol{\mu}) := \underset{v \in \mathcal{F}_{NN}}{\text{argmin}} \text{MSE}. \tag{8}$$

C. POD-NN hybrid approach

By combining the previous two methods, we obtain a promising paradigm: the POD-NN hybrid approach. This leverages the dimensionality reduction given by POD to obtain a compact representation of the solution space while using a neural network to learn the parametric dependence of the reduced coefficients. This strategy maintains the accuracy of POD while exploiting the benefits of neural networks, providing a computationally efficient alternative to both the MOR and PINN approaches.

POD-NN is a strategy that provides a fast tool to deal with non affine structures. It avoids the online projection by exploiting the direct projection of snapshots onto the reduced space built with the POD method, and then using them to create and train a neural network.

The general structure of the network can be summarized as follows:

$$\begin{aligned} \pi_{NN}: \quad & \mathcal{P} \to \mathbb{R}^N, \\ & \boldsymbol{\mu}^* \mapsto \boldsymbol{u}_N^{NN}(\boldsymbol{\mu}^*) \end{aligned} \tag{9}$$

where N is the dimension of the reduced space obtained by applying the POD. The inputs are the parameters of the training set, while the output is the Galerkin projection of the snapshots. For the output, which has the form $\boldsymbol{u}_N^{NN}$, the following relation holds:

$$\mathbb{B}\boldsymbol{u}_N^{NN}(\boldsymbol{\mu}) \approx \mathbb{P}^{\boldsymbol{\mu}} \boldsymbol{u}_\delta(\boldsymbol{\mu})$$

where $\mathbb{B} \in \mathbb{R}^{N_\delta \times N}$ is the basis functions matrix of the reduced space, and $\mathbb{P}^{\boldsymbol{\mu}} = \mathbb{B}\mathbb{P}_{\delta \to N} \in \mathbb{R}^{N_\delta \times N_\delta}$ is composed by the projection operator, which leads to the best approximation of u_δ in the reduced space, pre-multiplied by the basis function matrix. This allows us to compute $\boldsymbol{u}_N^{NN}(\boldsymbol{\mu})$ for each snapshot without solving the reduced system, obtaining the best approximation to $\boldsymbol{u}_\delta$ because we have a direct connection between $\boldsymbol{u}_\delta$ and $\boldsymbol{u}_N^{NN}(\boldsymbol{\mu})$. To train the network, we minimize the loss given by the distance between the exact u_N solution and the network approximation:

$$\text{MSE} = \frac{1}{N_{snap}} \sum_{\boldsymbol{\mu} \in \mathcal{P}_{train}} ||\boldsymbol{u}_N^{NN}(\boldsymbol{\mu}) - \boldsymbol{u}_N(\boldsymbol{\mu})||^2, \tag{10}$$

where N_{snap} is the number of data in the training set.

To sum up, we can compute offline the snapshots collection, apply the POD strategy and train the network. Then, we evaluate the network during the online phase. In this way, we exploit the POD properties and the online phase becomes very fast. In contrast, the offline phase is longer and the accuracy is more difficult to control.

III. Numerical studies

Introduzione con

- ◇ **'set up' di simulazione**
- ◇ **metriche usate per valutare risultati**
- ◇ **confronti tra gli esperimenti**

Esperimenti nello specifico: confronti risultati con grafici

A. Source term independent of parameters

B. Source term dependent on parameters

IV. Conclusions

References

[1] S. Berrone and C. Canuto, *Notes of the lecture course numerical methods for partial differential equations.* Non-published notes of the course on "Metodi numerici per le equazioni alle derivate parziali" at the Politecnico di Torino, 2024.

[2] F. Pichi, B. Moya, and J. S. Hesthaven, "A graph convolutional autoencoder approach to model order reduction for parametrized pdes," *Journal of Computational Physics*, vol. 501, p. 112762, 2024.